JURNAL PRAKTIKUM IOT

MODUL 1

KONEKTIVITAS BLUETOOTH LOW ENERGY (BLE)

Kelompok : .....

Hari/Shift : .....

Kode Asisten Pengajar : .....

Nama : .....

NIM : .....

PRAKTIKUM PEMROGRAMAN PERANGKAT IOT

FAKULTAS TEKNIK ELEKTRO

UNIVERSITAS TELKOM

2026

A. Rangkaian dan Sistem

1. Siapkan komponen berikut:

   - ESP32 DevKit V1 (DOIT) atau ESP32-S3 (2x)
   - Potensiometer 10 kΩ (1x)
   - Servo SG90 (1x)
   - Breadboard (2x)
   - Kabel/Jumper (disesuaikan)

2. Buatlah rangkaian berikut menggunakan daftar alat dan komponen yang telah kalian siapkan! (POT: Potensiometer, SRV: Servo)

Skema rangkaian Modul 1 (menyusul)

Central / Client		Peripheral / Server	
Komponen	**Pin**	**Komponen**	**Pin**
POT (kaki kiri)	GND	SRV (coklat)	GND
POT (wiper)	GPIO34 / GPIO4	SRV (merah)	VIN (5V)
POT (kaki kanan)	3V3	SRV (oranye)	GPIO16

Catatan: GPIO34 adalah pin masukan saja dan termasuk ADC1, satu-satunya ADC yang masih dapat dibaca selama radio BLE menyala. Servo SG90 menarik arus kejut yang tidak sanggup dipasok regulator 3V3, sehingga kabel merahnya wajib diambil dari pin VIN (5 V) dengan GND yang disatukan. Papan ESP32-S3 tidak memiliki ADC pada GPIO34, sehingga pada papan itu potensiometernya pindah ke GPIO4 dengan mengaktifkan `#define BOARD_ESP32S3` *pada program Central.*

Pembagian UUID per kelompok. Setiap kelompok memakai satu pasang UUID sendiri. Pada **kedua** program, hapuslah tanda `//` di depan pasangan milik kelompok kalian saja dan biarkan pasangan lain tetap dikomentari. Bila dua kelompok memakai pasangan yang sama, Central salah satu kelompok dapat tersambung ke Peripheral kelompok lain.

Kelompok	`SERVICE_UUID` *(4 karakter terakhir)*	`CHARACTERISTIC_UUID` *(4 karakter terakhir)*
1	`ce01`	`6501`
2	`ce02`	`6502`
3	`ce03`	`6503`
4	`ce04`	`6504`

Kelompok	`SERVICE_UUID` *(4 karakter terakhir)*	`CHARACTERISTIC_UUID` *(4 karakter terakhir)*
5	`ce05`	`6505`
6	`ce06`	`6506`
7	`ce07`	`6507`
8	`ce08`	`6508`

3. Sebelum menjalankan program, siapkan lingkungan kerja kalian dengan urutan berikut. Centanglah setiap langkah yang telah kalian selesaikan!

   - ☐ Memasang pustaka `ESP32Servo` melalui Library Manager Arduino IDE
   - ☐ Memilih board ESP32 Dev Module pada Arduino IDE untuk kedua papan
   - ☐ Mengaktifkan pasangan UUID kelompok kalian pada **kedua** program (lihat tabel di atas)
   - ☐ Menghapus tanda `//` pada `#define BOARD_ESP32S3` bila memakai papan ESP32-S3
   - ☐ Menyambungkan kabel merah servo ke VIN, bukan ke 3V3
   - ☐ Menyinkronkan jam **kedua** papan dari PC yang sama: `.\scripts\send_time.ps1 -Port COM_A,COM_B`

4. Lengkapi kode program di bawah ini dengan mengisikan bagian-bagian yang rumpang. Selanjutnya (atau sambil melengkapi), ketiklah kembali bagian-bagian rumpang yang telah diisi pada aplikasi VS Code/Arduino IDE yang telah disiapkan oleh asisten jaga kalian masing-masing!

   Bagian rumpang ditandai dengan garis bawah _____ dan diberi nomor `(1)` sampai `(6)` pada komentar di sebelah kanannya. Gantilah setiap garis bawah tersebut dengan kode yang benar sesuai petunjuk pada komentarnya. Ingat bahwa UUID service dan UUID characteristic pada sisi Central harus sama persis dengan nilai pada sisi Peripheral.

Central/Client Source Code — Bluetooth Low Energy (ESP32)

```
 */

#include <BLEDevice.h>
#include <BLEUtils.h>
#include <________>  // (1) Berkas header pemindai (scan) perangkat BLE
#include <BLEAdvertisedDevice.h>
#include <time.h>

// ---- UUID KELOMPOK -------------------------------------------------------
…
}

// Mengubah tegangan pada potensiometer menjadi sudut servo.
int readAngle() {
  int raw = ________(POT_PIN);  // (2) Fungsi pembaca nilai analog sebuah pin

  return map(raw, 0, ADC_MAX, SERVO_MIN_ANGLE, ________);  // (3) Batas sudut tertinggi
}

// Dipanggil pustaka BLE setiap kali Peripheral mengirim notifikasi. Di sinilah
// pantulan (echo) diterima, sehingga di sinilah delay pulang-pergi dihitung.
…
    return false;
  }

  // Service adalah wadah, characteristic adalah data di dalamnya.
  BLERemoteService *pService = pClient->getService(________);  // (4) UUID service

```

```
214     if (pService == nullptr) {
215       Serial.println("Service tidak ditemukan!");
216       pClient->disconnect();
…
232     }
233
234     // Tanpa pendaftaran ini, pantulan dari Peripheral tidak akan pernah sampai
235     // dan kolom Delay pada jurnal tidak dapat diisi.
236     pRemoteCharacteristic->registerForNotify(________);  // (5) Nama fungsi notifikasi
237
238     connected = true;
239     Serial.println("Siap! Putar potensiometer untuk menggerakkan servo.");
240     return true;
…
305       lastSendTime = sendTime;
306       lastSentAngle = angle;
307       waitingForEcho = true;
308
309       String payload = String(________);  // (6) Variabel sudut yang ingin dikirim
310
311       Serial.printf("[%s] Potensiometer berubah, mengirim sudut: %s derajat\n",
312                     formatTimestamp(sendTime).c_str(), payload.c_str());
```

Peripheral/Server Source Code — Bluetooth Low Energy (ESP32)

```
10   * adalah petunjuknya.
11   */
12
13  #include <BLEDevice.h>
14  #include <________>  // (1) Berkas header pembuat server/peripheral BLE
15  #include <BLEUtils.h>
16  #include <BLE2902.h>
17  #include <ESP32Servo.h>
18  #include <time.h>
…
154     // Memantulkan data secepat mungkin, sebelum menggerakkan servo dan mencetak
155     // ke Serial, agar pengukuran pulang-pergi di sisi Central tidak terbebani.
156     pCharacteristic->setValue(data.c_str());
157     pCharacteristic->________();  // (2) Metode pengirim nilai ke Central
158
159     servo.write(________);  // (3) Variabel sudut hasil pembacaan data
160
161     Serial.printf("[%s] Sudut dari Central: %d derajat | Servo digerakkan\n",
162                   formatTimestamp(receiveTime).c_str(), angle);
163   }
…
178   // PROPERTY_WRITE_NR -> Central mengirim data tanpa konfirmasi
179   pCharacteristic = pService->createCharacteristic(
180     BLEUUID(CHARACTERISTIC_UUID),
181     BLECharacteristic::PROPERTY_WRITE |
182     BLECharacteristic::________  // (4) Properti pengirim data otomatis tanpa ACK
183   );
184
185   // Descriptor 0x2902 inilah yang mengizinkan Central menghidupkan notifikasi.
186   pCharacteristic->addDescriptor(new BLE2902());
187   pCharacteristic->setCallbacks(new ________());  // (5) Nama kelas callback tulisan
188   pCharacteristic->setValue("0");
189
190   pService->start();
…
201 void setup() {
202   Serial.begin(115200);
203   syncTimeFromSerial();
204
205   servo.attach(________, SERVO_MIN_US, SERVO_MAX_US);  // (6) Konstanta pin servo
206   servo.write(90);
207
208   BLE_init();
209 }
```

5. Catatlah konfigurasi yang kalian pakai pada tabel berikut ini! Kedua UUID inilah yang harus sama persis antara sisi Central dan sisi Peripheral agar koneksi dapat terbentuk, karena BLE menyaring lawan bicaranya berdasarkan UUID service, bukan berdasarkan nama perangkat.

Parameter	**Nilai yang dipakai**
Nama perangkat Central (`centralName`)	
Nama perangkat Peripheral (`peripheralName`)	
Nomor kelompok dan pasangan UUID-nya	
`SERVICE_UUID` *(4 karakter pertama saja)*	
`CHARACTERISTIC_UUID` *(4 karakter pertama saja)*	
Pin potensiometer pada Central (`POT_PIN`)	
Pin servo pada Peripheral (`SERVO_PIN`)	
Ambang perubahan sudut (`DEADBAND_DEG`)	

B. Analisis

1. Gambarkanlah alur komunikasi yang diterapkan pada sistem yang telah kalian kerjakan!

2. Selanjutnya akan dilakukan percobaan respons waktu pengiriman data dari **Central/Client** ke **Peripheral/Server**. Bukalah terlebih dahulu **DUA (2)** Serial Monitor untuk memantau kondisi keluaran dari **Central/Client** dan **Peripheral/Server**. Setiap kali potensiometer diputar cukup jauh, Serial Monitor Central mencetak **waktu kirim** beserta sudut yang dikirim, lalu mencetak delay **round-trip** begitu pantulan (echo) dari Peripheral diterima; sementara Serial Monitor Peripheral mencetak **waktu terima** beserta sudut servo yang diterapkan. Catatlah:
   - **LIMA (5)** pasang data: waktu kirim dan sudut dari Serial Monitor Central, beserta waktu terima dan sudut servo dari Serial Monitor Peripheral

   Kedua waktu tersebut dicatat dalam format **HH:MM:SS:MS**. Hitunglah **delay satu arah** dengan mengurangkan waktu terima terhadap waktu kirim, lalu bandingkan dengan delay round-trip yang dicetak sendiri oleh Central.

   *Catatan: kedua papan tidak memiliki akses Internet, sehingga jamnya diselaraskan dari jam PC melalui Serial. Selama **kedua papan disinkronkan dari PC yang sama** (jalankan `send_time.ps1` dengan dua*

*COM port sekaligus), jam keduanya sama sampai ke orde milidetik dan waktu dari dua Serial Monitor yang berbeda **boleh** dikurangkan. Ketelitian sinkronisasi lewat Serial berkisar beberapa milidetik. Bila kedua papan disinkronkan dari PC yang berbeda, atau salah satunya gagal menerima waktu PC (tampil pesan "waktu sejak boot"), kolom delay satu arah menjadi tidak sahih dan hanya kolom round-trip yang dapat dipakai.*

Percobaan ke-…	Sudut servo **[derajat]**	Waktu kirim *(Central)* **[HH:MM:SS:MS]**	Waktu terima *(Peripheral)* **[HH:MM:SS:MS]**	Delay satu arah **[ms]**	Delay round-trip **[ms]**
1.					
2.					
3.					
4.					
5.					
Rata-rata delay (ms)					

3. Tuliskanlah **analisis** kalian pada sistem yang telah kalian eksperimentasikan sebelumnya pada bagian pengisian berikut ini!

(Berikan juga **hubungan/korelasinya** dengan menerapkan **metodologi komunikasi** yang sedang kalian pelajari pada modul ini; tidak harus penuh!)

C. Saran dan Masukan

1. Asisten

2. Laboratorium